

Curso de Especialización Ciberseguridad en las TIC

Ejercicio 6

Objetivos

- Analizar el tráfico web.
- Detectar datos sensibles y malas prácticas observando tráfico HTTP.
- Relacionar fallos reales con el OWASP Top 10.

Resumen

Autora: Marina del Pilar López Oliva

Febrero 2026

IES Camp de Morvedre

Obra publicada con [Licencia Creative Commons Reconocimiento Compartir igual 4.0](https://creativecommons.org/licenses/by-sa/4.0/)



Revisión

Revisión	Fecha	Descripción
1.0	11/02/2026	Realización del ejercicio.

Índice de Contenidos

ACTIVIDAD 2 – Análisis de tráfico web (nivel básico-medio).....	4
ACTIVIDAD 3 – Detección de vulnerabilidades OWASP.....	4

ACTIVIDAD 2 – Análisis de tráfico web (nivel básico-medio)

En la captura proporcionada se observa una petición. Se utiliza el método POST, lo cual es adecuado para el envío de credenciales, ya que los datos no se transmiten en la URL como ocurría con GET. Sin embargo, el uso correcto del método no nos asegura la seguridad de la aplicación.

La petición se realiza mediante HTTP, no HTTPS, por lo que no existe cifrado de la comunicación, no hay autenticación del servidor y no se garantiza la integridad de los datos. El tráfico viaja en texto plano, por lo que cualquier atacante en la misma red puede capturarlo con herramientas como Wireshark y leer su contenido.

En el cuerpo de la petición, se envían las credenciales. Se tratan de credenciales administrativas enviadas en texto plano, lo que representa una exposición crítica de información sensible.

Además, la captura revela el endpoint de autenticación y la estructura del sistema de autenticación. Esta información facilita posibles ataques dirigidos.

Las credenciales viajan en texto plano por la red, exponiéndose a riesgos como sniffing pasivo de red, ataques Man-in-the-Middle (MiTM), robo de credenciales y accesos no autorizados.

El uso de credenciales débiles como el usuario por defecto y la contraseña débil y predecible, puede ser riesgoso para ataques de fuerza bruta, ataques de diccionario y credential stuffing.

Si capturáramos este tráfico con Wireshark, veríamos algo así:

```
<!-- END FOOTER -->
POST /doLogin HTTP/1.1
Host: demo.testfire.net
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Content-Type: application/x-www-form-urlencoded
Content-Length: 41
Origin: http://demo.testfire.net
Connection: keep-alive
Referer: http://demo.testfire.net/login.jsp
Cookie: JSESSIONID=A367D909DB090B727656C917F2B91725
Upgrade-Insecure-Requests: 1
Priority: u=0, i

uid=admin&passw=admin1234&btnSubmit=Login
HTTP/1.1 302 Found
Server: Apache-Coyote/1.1
Location: login.jsp
Content-Length: 0
Date: Wed, 11 Feb 2026 15:36:17 GMT
```

Una vez corregido con HTTPS quedaría así, con el contenido ilegible:



Esta captura representa un riesgo crítico de seguridad. Las credenciales administrativas están completamente expuestas a cualquier atacante que pueda interceptar el tráfico de red.

La implementación de HTTPS es imprescindible para proteger la confidencialidad de las comunicaciones, especialmente en procesos de autenticación.

ACTIVIDAD 3 – Detección de vulnerabilidades OWASP

Vulnerabilidad	Categoría OWASP	Descripción	Corrección
Control de acceso solo en frontend	A01:2021 - Broken Access Control	El panel de administración valida permisos únicamente mediante JavaScript en el cliente.	Implementar validación de roles y permisos en backend.
Inyección SQL	A03:2021 - Injection	La API concatena directamente input del usuario queries SQL sin sanitización.	Usar consultas parametrizadas, validar y sanitizar toda entrada del usuario.
Tokens JWT sin fecha de caducidad	A07:2021 - Identification and Authentication Failures	Los tokens se generan sin campo exp de expiración. Una vez obtenido un token, es válido siempre..	Establecer tiempo de expiración corto.
Campos extra en JSON	A04:2021 - Insecure Design	La API acepta cualquier campo JSON en la actualización de perfil	Implementar whitelist de campos permitidos.

Ejercicio 6

Exposición de información sensible en mensajes de error	A05:2021 – Security Misconfiguration	Los errores de login revelan si el usuario existe: "Contraseña incorrecta" vs "Usuario no encontrado" Los errores de API exponen stack traces en producción con rutas del servidor y versiones de librerías.	Usar mensajes genéricos: "Credenciales incorrectas".
Falta de rate limiting en endpoints de login	A07:2021 – Identification and Authentication Failures	No hay límite de intentos de login.	Implementar rate limiting. Implementar CAPTCHA después de 3 intentos fallidos. Bloqueo temporal de cuenta tras 10 intentos. Monitoreo y alertas de intentos masivos